

Queues Crash Course

Apache Kafka: Ecosystem



In the Part II:

- Stream processing and EventSourcing with Kafka Streams
- Kafka Connect
- Schema Registry
- Cross-Cluster Data Replication with MirrorMaker

Kafka Streams and KSQL

Stream processing and EventSourcing

Processor Topology Overview

- Directed Acyclic Graph
- Defines the flow of data through stream processing nodes.

Processor Topology Overview

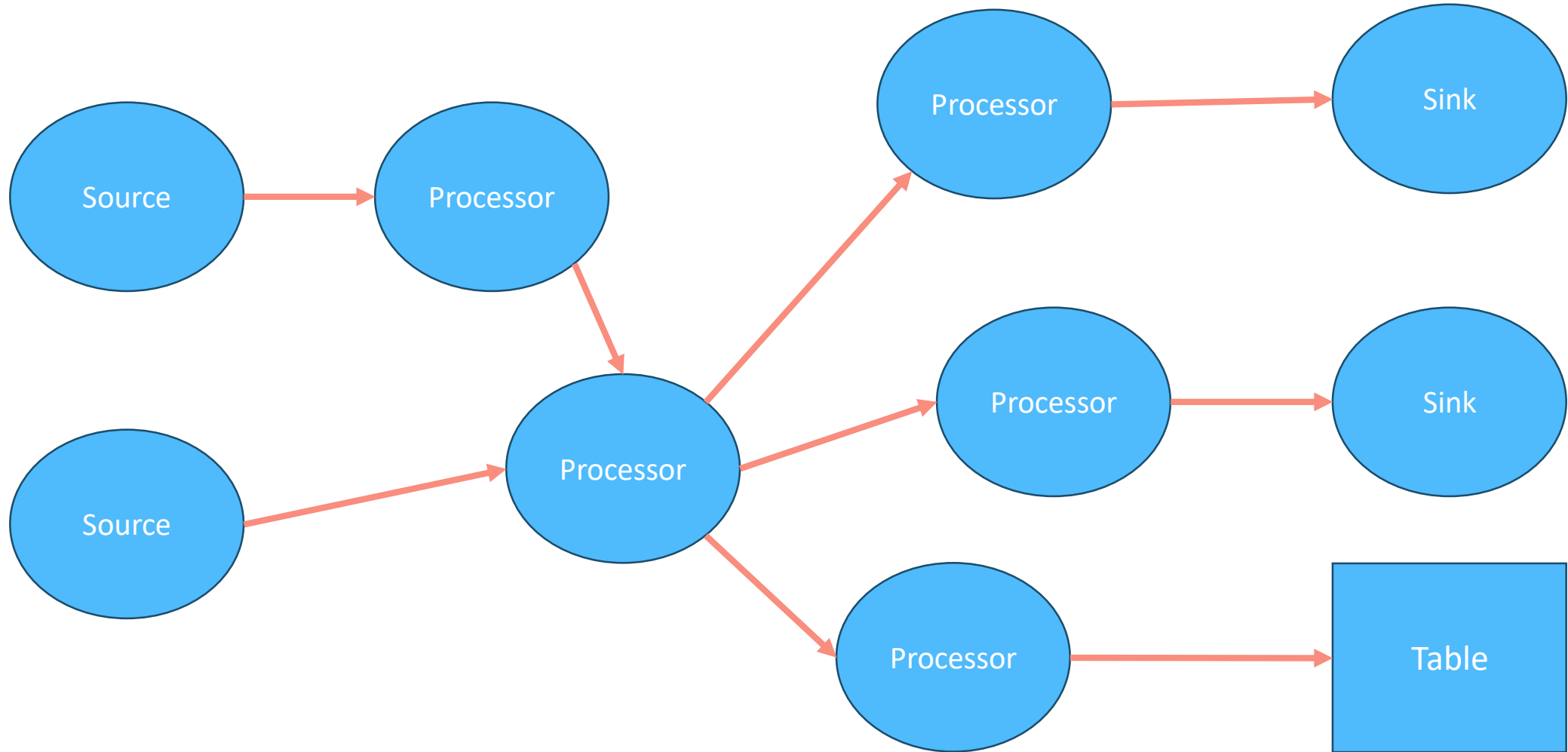
Components

- **Sources:** Nodes that read data from Kafka topics.
- **Processors:** Nodes that apply operations to transform the data.
- **Sinks:** Nodes that write the processed data back to Kafka topics.

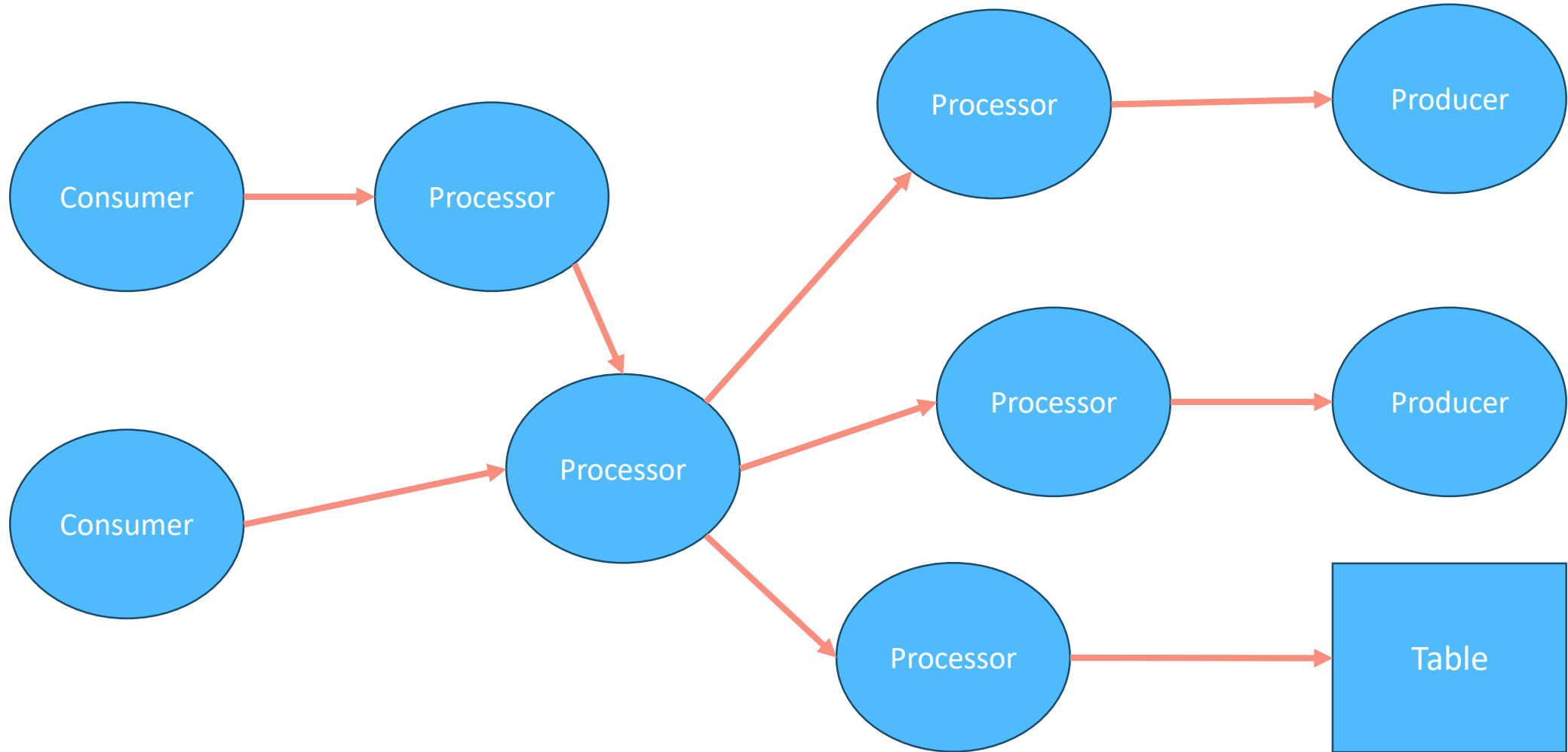
Example

- A source reads data from the "input" topic.
- A processor filters and transforms the data.
- A sink writes the transformed data to the "output" topic.

Processor Topology



Processor Topology



Kafka Streams is Java App

Implementing source + processor + sink

Example Topology Code

```
builder = new StreamsBuilder();  
sourceStream = builder.stream("input-topic");
```

Example Topology Code

```
builder = new StreamsBuilder();  
sourceStream = builder.stream("input-topic");  
  
sourceStream  
    .filter((key, value) -> value.contains("filter"))
```

Example Topology Code

```
builder = new StreamsBuilder();  
sourceStream = builder.stream("input-topic");  
  
sourceStream  
    .filter((key, value) -> value.contains("filter"))  
    .mapValues(value -> value.toUpperCase())
```

Example Topology Code

```
builder = new StreamsBuilder();  
sourceStream = builder.stream("input-topic");  
  
sourceStream  
    .filter((key, value) -> value.contains("filter"))  
    .mapValues(value -> value.toUpperCase())  
    .to("output-topic");
```

KStream

- Abstraction for a continuous stream of records.
- Each record is a key-value pair.
- Represents an unbounded dataset.
- Supports stateless and stateful operations.

```
KStream<String, String> stream =  
    builder.stream("topic");
```

Operations on KStream

- Peek
 - Peeking at the records as they pass through the stream.
- Map and MapValues (transform)
 - *map*: transforms each record into a new record with potentially different key and value.
 - *mapValues*: transforms only values.
- Filter
 - Removes records from stream by condition.

Operations on KStream

```
kstream.peek((key, value) -> println(key, value))
```

```
kstream.map((key, value) ->  
    KeyValue.pair(key.toUpperCase(), value.toUpperCase()))
```

```
kstream.mapValues(value -> value.toUpperCase())
```

```
kstream.filter((key, value) -> value.contains("filter"))
```

KTable

- Abstraction of a changelog stream, where each record represents an update.
- Represents a snapshot of data at a given time.
- Supports stateful operations.
- Uses local storage for maintaining state.

```
KTable<String, String> table =  
    builder.table("input-topic");
```


KTable

- Consumes single topic
 - Do not stream results
 - Needs materialized state store
 - The cache inside: flush every 30s
-
- KTable: works on topic partition
 - GlobalKTable: works on the whole topic

KTable



KTable



KTable



KTable



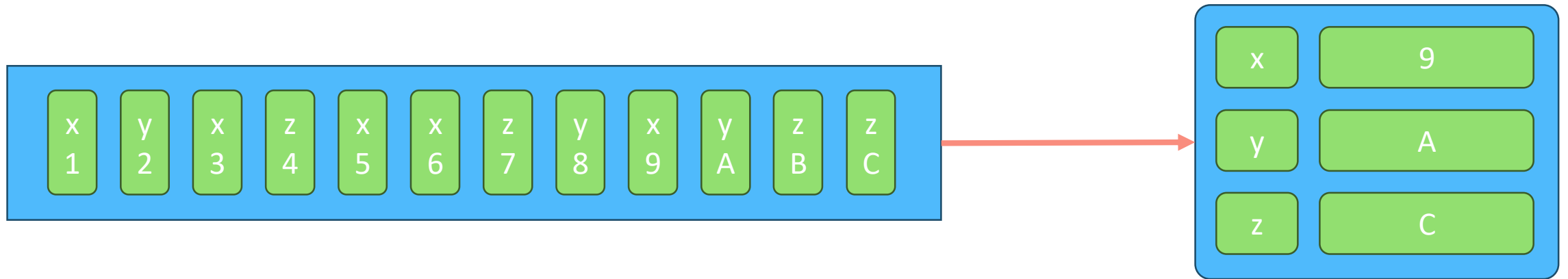
KTable



KTable



KTable



Joins

- Stream-Stream
 - 2 events → 1 new event
 - join based on common key
 - Join waits for a given “window” time
- Stream-Table
 - Stream-Table
 - Stream-GlobalTable
- Table-Table

Joins example

```
KStream<String, String> joined =  
    stream1.join(  
        stream2,  
        (left, right) -> left + right,  
        JoinWindows.of(Duration.ofMinutes(5))  
    );
```

Types of joins for Stream-Stream

- Inner
 - Produces result only if have left and right
- Outer
 - Left + Right
 - Left + Null
 - Null + Right
- LeftOuter
 - Left + Right
 - Left + Null

Types of joins for Stream-Table

- No windowed (Table query is immediate)
- Inner
 - Produces result only if have left and right
- LeftOuter
 - Left + Right
 - Left + Null
- Stream always Left (triggers event output)
- KTables are timestamp driven
- GlobalKTables are bootstrapped (read all at start)

Types of joins for Table-Table

- No windowed (Table query is immediate)
- The result is ***KTable***, not Kstream
- Types:
 - Inner
 - Outer
 - LeftOuter

Stateful operations (aggregations)

- Operations that maintaining state
- Need info from previous events
- Like tables they need Materialized
- Operations:
 - aggregate
 - reduce
 - count

Stateful operations (aggregations)

```
stream.groupByKey().count();
```

```
stream
    .groupByKey()
    .aggregate(
        () -> 0L,
        (key, value, agg) -> agg + value.length()
    );
```

Aggregation windowing

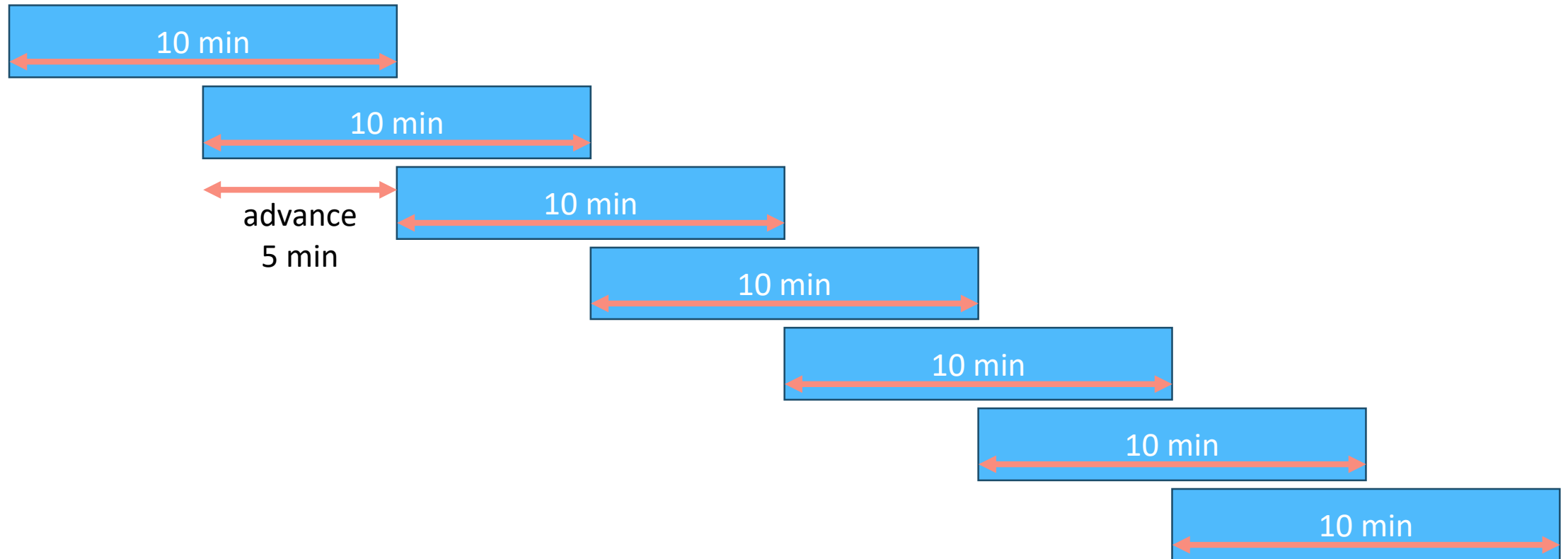
- Windowing is a technique to group records based on time intervals.
- Kinds:
 - Tumbling
 - Hopping
 - Sliding
 - Session

Stateful operations (aggregations)

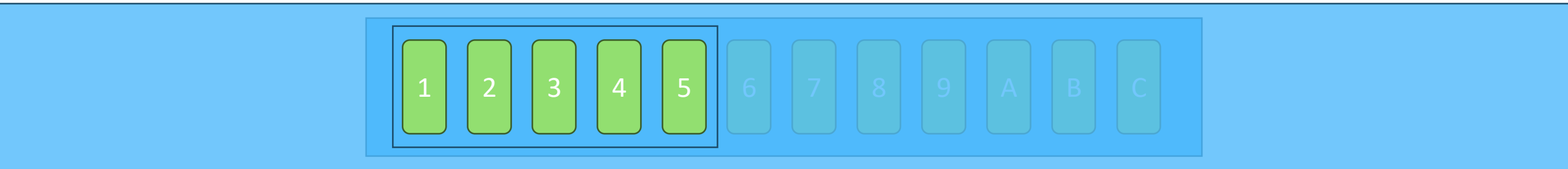
```
stream.groupByKey().count();
```

```
stream
    .groupByKey()
    .windowedBy(Timewindows.of(Duration.ofMinutes(5)))
    .aggregate(
        () -> 0L,
        (key, value, agg) -> agg + value.length()
    );
```

Hopping window



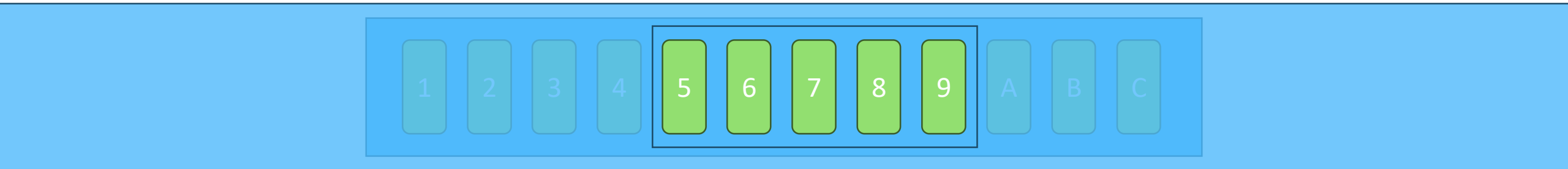
Hopping window



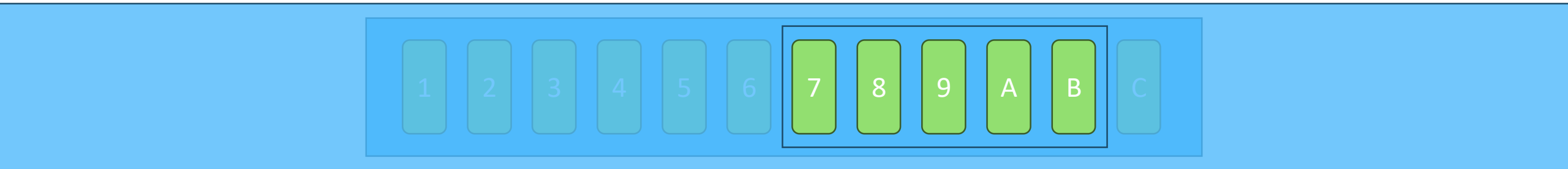
Hopping window



Hopping window

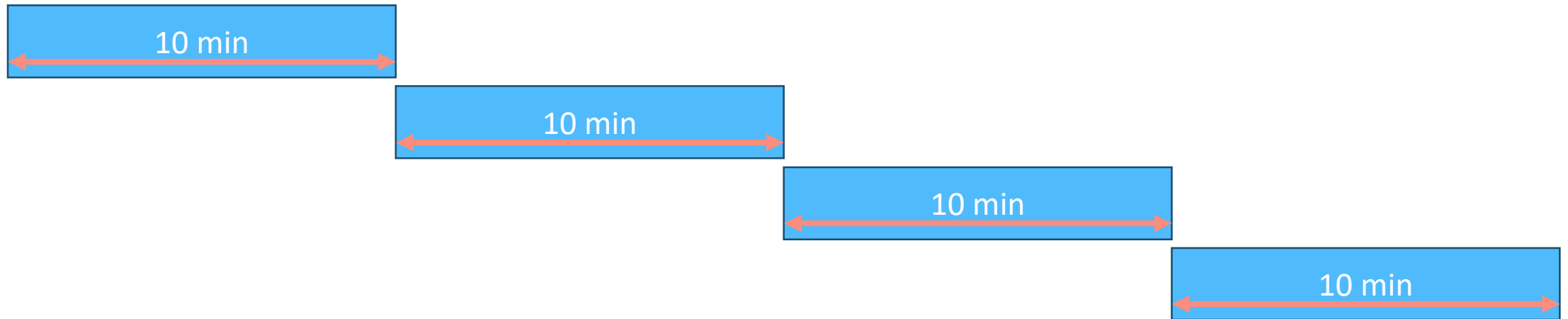


Hopping window

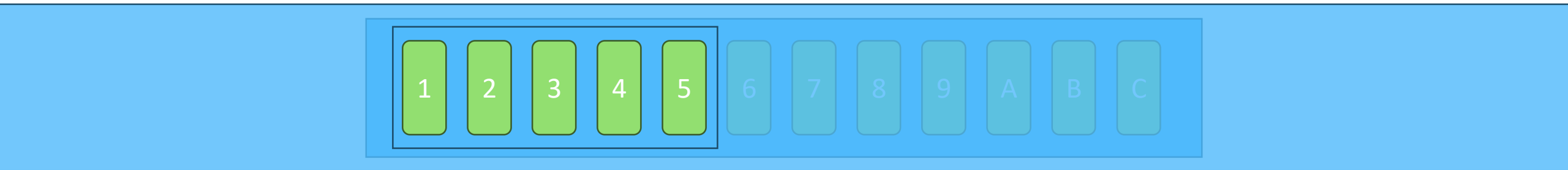


Thumbling window

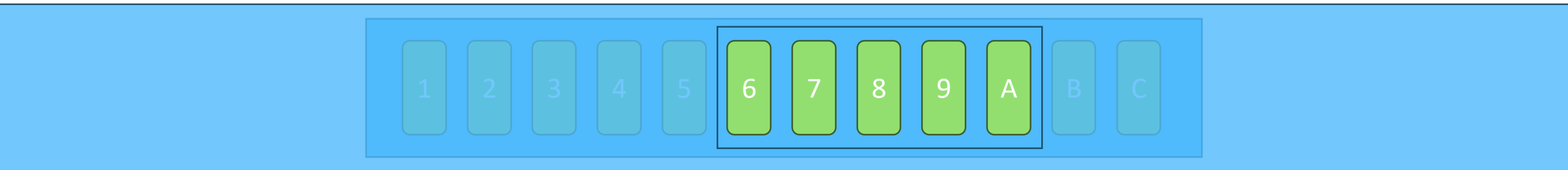
advance = window size



Tumbling window



Tumbling window



Thumbling window

```
stream
    .groupByKey()
    .windowedBy(
        Timewindows
            .of(Duration.ofMinutes(10))
    )
    .count();
```

Hopping window

```
stream
```

```
    .groupByKey()
```

```
    .windowedBy(
```

```
        TimeWindows
```

```
            .of(Duration.ofMinutes(10))
```

```
            .advanceBy(Duration.ofMinutes(5))
```

```
    )
```

```
    .count();
```

Sliding window

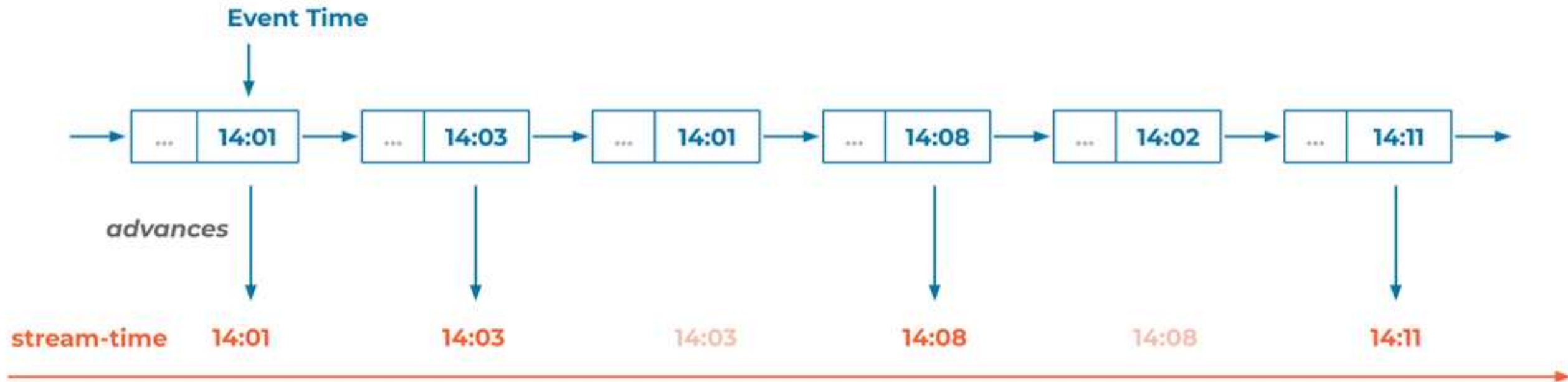
```
stream
    .groupByKey()
    .windowedBy(
        SlidingWindows
            .ofTimeDifferenceWithNoGrace(
                Duration.ofMinutes(5)
            )
    )
    .count();
```

Session window

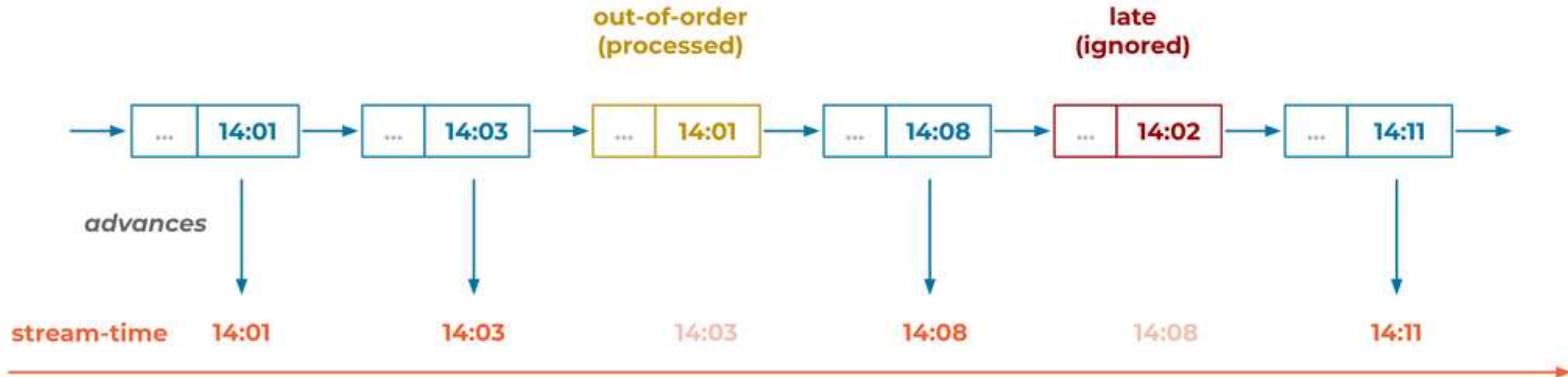
stream

```
.groupByKey()  
.windowedBy(  
    SessionWindows.with(Duration.ofMinutes(5))  
)  
.count();
```

Stream Time



Grace period



Processor API

- Low-level API for building custom stream processing logic.
- When the DSL does not provide the required functionality.
- Processor API offers more flexibility than the DSL.
- Allows for fine-grained control over processing logic.
- Can be integrated with the DSL.

Durability of Local State and Failover

- Local storage used for maintaining state.
- Local storage may be in-memory or disk based.

Durability of Local State and Failover

- Local storage used for maintaining state.
- Local storage may be in-memory or disk based.
- Processor node maintains a changelog of state in special topic
- Changelog is compacted to allow faster recovery
- Upon failure, the whole state may be reconstructed from changelog

Durability of Local State and Failover

- Local storage used for maintaining state.
- Local storage may be in-memory or disk based.
- Processor node maintains a changelog of state in special topic
- Changelog is compacted to allow faster recovery
- Upon failure, the whole state may be reconstructed from changelog
- To speedup failover, there are standby processor nodes
- Standby nodes only consume changelogs and maintain state

Durability of Local State and Failover

- Local storage used for maintaining state.
- Local storage may be in-memory or disk based.
- Processor node maintains a changelog of state in special topic
- Changelog is compacted to allow faster recovery
- Upon failure, the whole state may be reconstructed from changelog
- To speedup failover, there are standby processor nodes
- Standby nodes only consume changelogs and maintain state

ksqlDB

Kafka Streams with SQL

ksqlDB: Kafka Streams with SQL

- KSQL is a SQL-based stream processing engine for Apache Kafka.
- Simplifies data processing by writing streaming queries using SQL.
- Easy to use with SQL-like syntax.
- No need for complex coding.
- Real-time continuous query execution.
- By Confluent

Basic KSQL Queries: Create stream

```
CREATE STREAM pageviews(  
    viewtime BIGINT,  
    userid VARCHAR,  
    pageid VARCHAR  
)  
WITH (  
    kafka_topic='pageviews',  
    value_format='DELIMITED'  
);
```

Basic KSQL Queries: selecting and filtering

```
SELECT * FROM pageviews  
WHERE pageid = 'home'  
EMIT CHANGES;
```


Advanced KSQL Queries: Transformations

```
SELECT
    userid,
    UCASE(pageid) AS upper_pageid
FROM pageviews
EMIT CHANGES;
```

Advanced KSQL Queries: Tables

```
CREATE TABLE user_profiles (  
    userid VARCHAR,  
    regionid VARCHAR,  
    gender VARCHAR  
)  
WITH (  
    kafka_topic='user_profiles',  
    value_format='JSON',  
    key='userid'  
);
```

Advanced KSQL Queries: Joins

```
SELECT
    p.viewtime, p.userid,
    p.pageid, u.regionid, u.gender
FROM pageviews p
LEFT JOIN user_profiles u ON p.userid = u.userid
EMIT CHANGES;
```

Aggregations and Windowing in KSQL

```
SELECT
    pageid,
    COUNT(*) AS view_count
FROM pageviews
GROUP BY pageid
EMIT CHANGES;
```

Windowed Aggregation

```
CREATE TABLE pageview_counts AS
  SELECT
    pageid,
    COUNT(*) AS view_count
  FROM pageviews
  WINDOW TUMBLING (SIZE 1 HOUR)
  GROUP BY pageid;
```

Session Windowing

```
SELECT
    userid,
    COUNT(*) AS session_count
FROM pageviews
WINDOW SESSION (5 MINUTES)
GROUP BY userid
EMIT CHANGES;
```

What if not Java and Scala

Alternatives

- Apache Flink
 - Stateful Computations over Data Streams
 - Advanced, stateful, scalable processing
- Apache Beam
 - SDKs for defining and executing data processing workflows
 - Unified batch/stream model
- Materialize
 - Real-time materialized views

Apache Flink

Distributed stream and batch data processing framework

- Standalone distributed system
- Scale independently of Kafka
- Flink can connect to many systems, not just Kafka
- Flink jobs are not inherently tied to Kafka
- Flexible for other data pipelines

Pure programming

- Generic Producer/Consumer API for any language + State Storage
- Python:
 - Faust (*python-native stream processing library*)
 - Confluent Kafka client
- Golang:
 - Kafka-Go
 - Watermill
 - A Go library for working efficiently with message streams. It's not specific to Kafka but has a Kafka adapter.

Kafka Connect

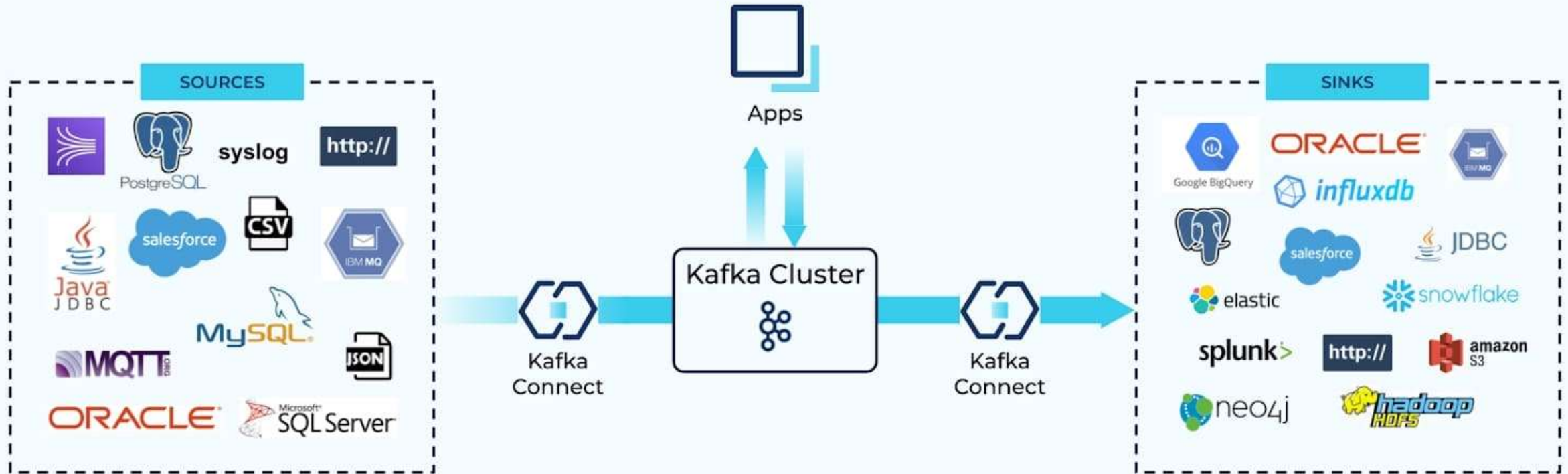
What is Kafka Connect?

- Framework for connecting Kafka with external systems
 - databases
 - key-value stores
 - search indexes
 - datalakes and storages
 - file systems
- Purpose
 - Simplifies data integration and stream processing.
 - Provide scalable and reliable data pipelines.

Benefits and Use Cases

- **Benefits:**
 - Easy to set up and configure.
 - Scalable and fault-tolerant.
 - Supports a wide range of connectors.
- **Use Cases:**
 - Data migration between systems.
 - Real-time analytics.
 - ETL processes.

Ingest Data from Upstream Systems

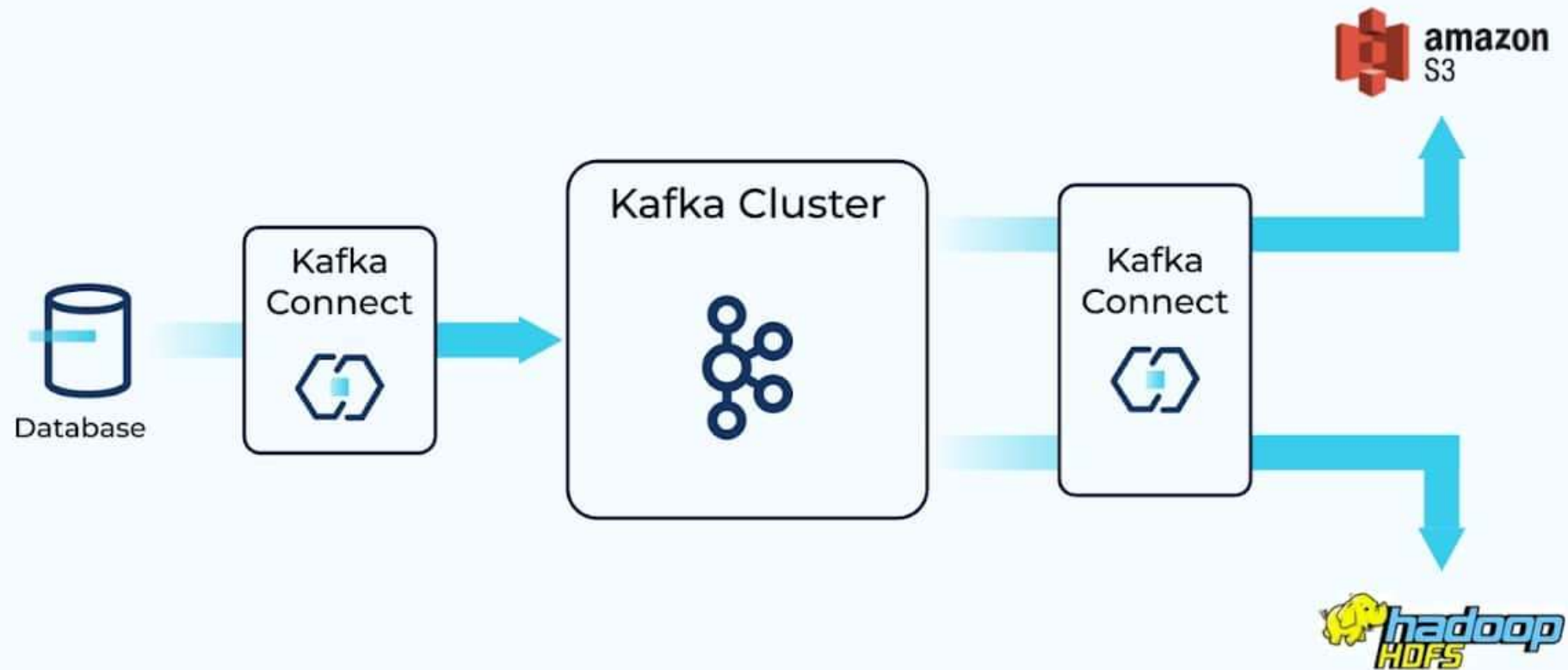


How Kafka Connect Works



```
{  
  "connector.class":  
    "io.confluent.connect.jdbc.JdbcSourceConnector",  
  "connection.url":  
    "jdbc:mysql://asgard:3306/demo",  
  "table.whitelist":  
    "sales,orders,customers"  
}
```

Streaming Pipelines

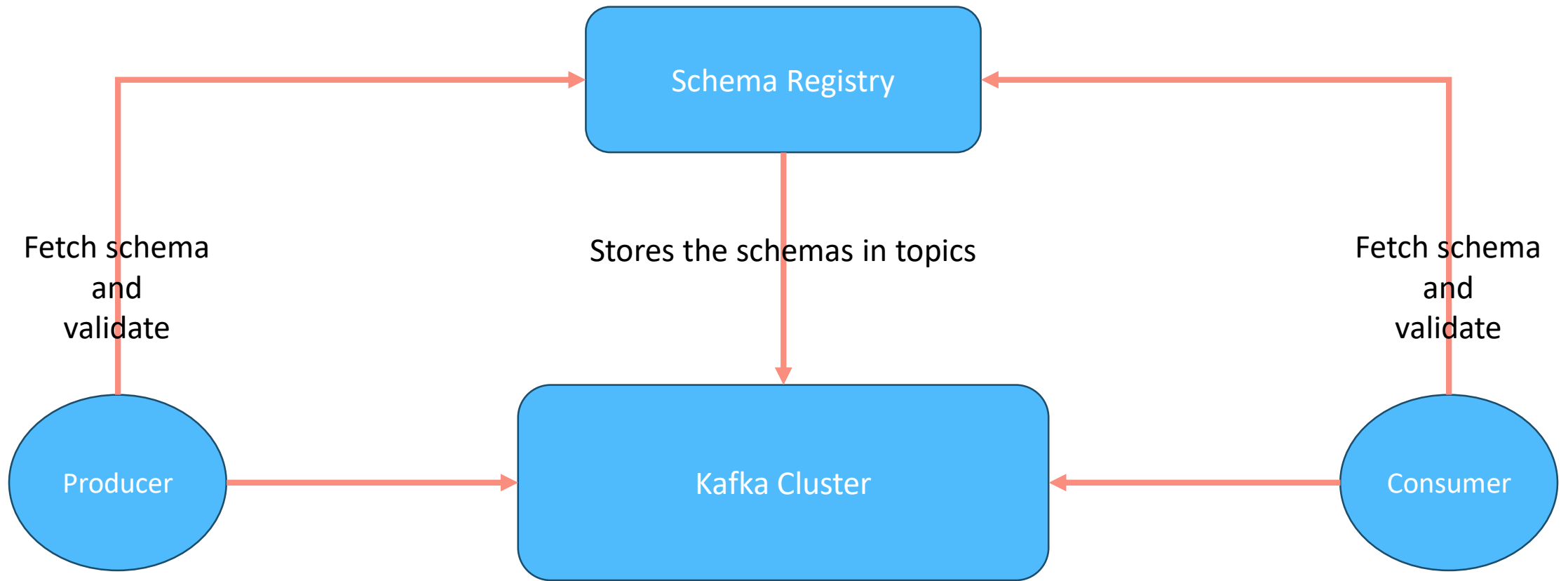


Schema Registry

What is Schema Registry?

- Schema Registry is a service that provides a central repository for managing and validating schemas for Kafka messages.
- Ensures that data is serialized and deserialized consistently across producers and consumers.
- Defines the structure of data (e.g., Avro, JSON Schema, Protobuf).
- Logical groupings of schemas, typically tied to a Kafka topic.
- Schemas can evolve, and each change is tracked as a new version.
- Uses Kafka Cluster as a backing store.

Schema Registry



Define the avro schema for event

```
{  
  "namespace": "io.devhands.avro",  
  "type": "record",  
  "name": "Event",  
  "fields": [  
    {"name": "name", "type": "string"},  
    {"name": "kind", "type": "string"},  
    {"name": "data", "type": "long"}  
  ]  
}
```

Publish the schema into registry

```
curl -X POST -H "Content-Type:
application/vnd.schemaregistry.v1+json" \
--data '{"schema": "{\\"type\\": \\"record\\", \\"name\\":
\\"Event\\", \\"fields\\": [{\\"name\\": \\"name\\", \\"type\\":
\\"string\\"}, {\\"name\\": \\"kind\\", \\"type\\": \\"string\\"},
{\\"name\\": \\"data\\", \\"type\\": \\"long\\"}]}"}' \
http://kafka-registry:8081/subjects/event-avro/versions
```

Fetch the schema from registry

```
curl -X GET http:// kafka-registry:8081/subjects/event-  
avro/versions/latest
```

Schema Evolution and Compatibility

The process of modifying a schema over time while ensuring compatibility with existing data.

- Types of Changes:
 - **Backward**: New schema can read data written by the old schema.
 - **Forward**: Old schema can read data written by the new schema.
 - **Full Compatible**: Both backward and forward compatible.
- Backward is the default

```
curl -XPUT --data '{"compatibility":"BACKWARD"}' \
http://.../config/event-avro
```

Best Practices for Using Schema Registry

Schema Design:

- Design schemas with forward and backward compatibility in mind.
- Use defaults for new fields to maintain compatibility.

Versioning:

- Increment schema versions for changes.
- Use meaningful versioning conventions.

Compatibility:

- Regularly check compatibility mode settings.
- Use full compatibility for the highest level of data integrity.

Best Practices for Using Schema Registry

Documentation:

- Document schema changes and evolution history.
- Maintain a changelog for schema updates.

Security:

- Secure access to the Schema Registry API.
- Use authentication and authorization to control access.

Kafka Mirroring

How to live with only ~~one~~ leg two datacenters

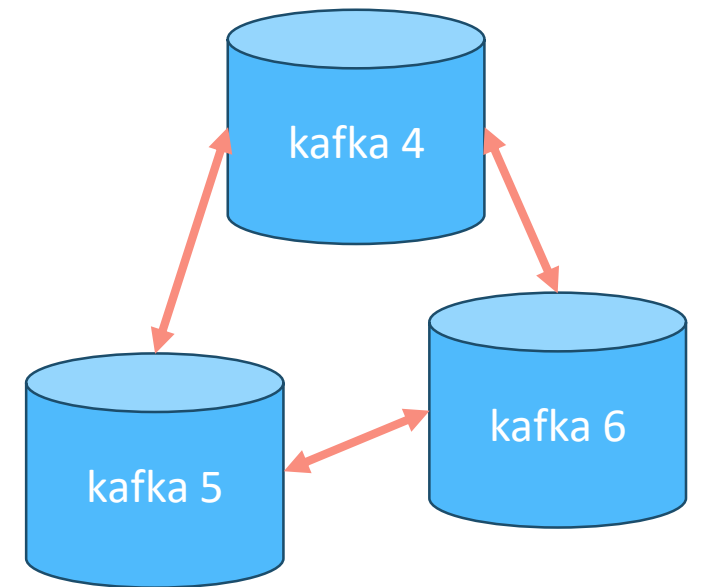
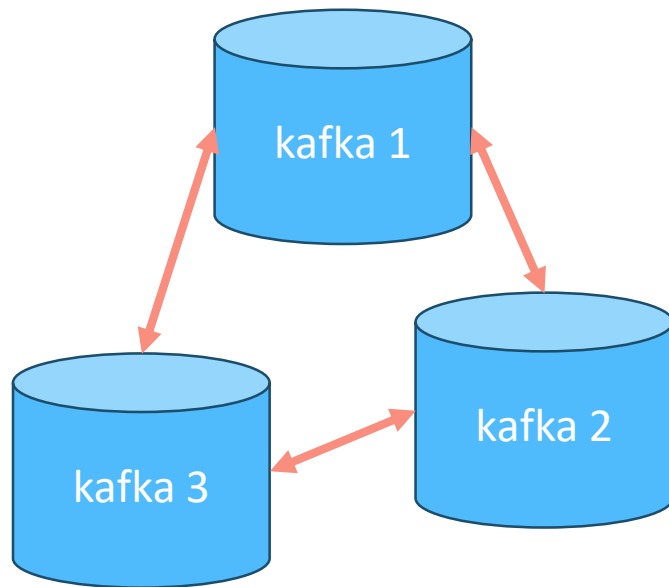
What is Kafka MirrorMaker?

- A tool provided by Apache Kafka for cross-cluster replication.
- Designed to replicate topics between clusters.
- Common use cases:
 - Disaster recovery and backup.
 - Geo-replication for multi-region Kafka clusters.
 - Migration between Kafka clusters.

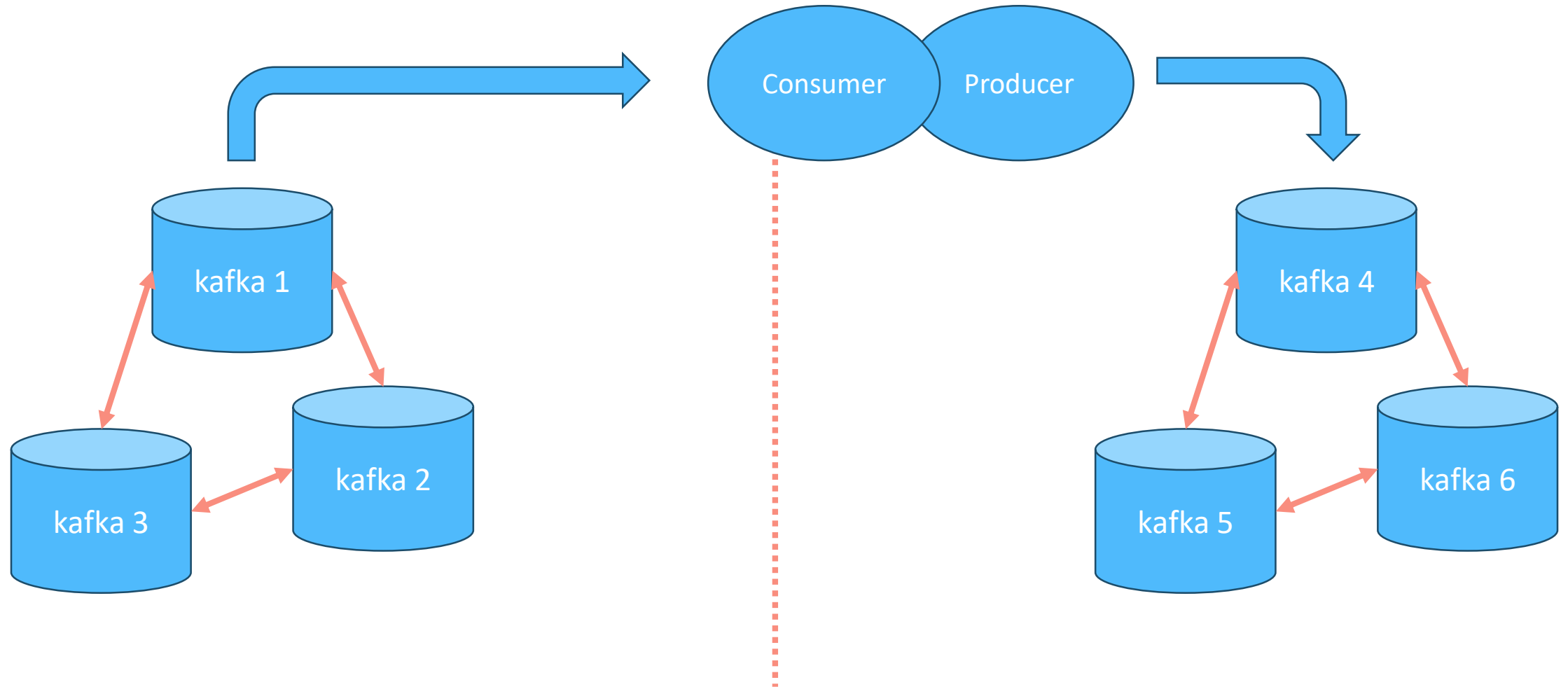
MirrorMaker 2 (MM2)

- MM2 introduced with Kafka 2.4
- MM2 is a major upgrade, built on top of Kafka Connect.
- Key improvements over the original MirrorMaker:
 - Based on Kafka Connect framework.
 - Better configurability and modularity.
 - Supports active-active replication between clusters.
 - Enhanced monitoring and metrics support.
 - Handles topic renaming and automatic offset sync.

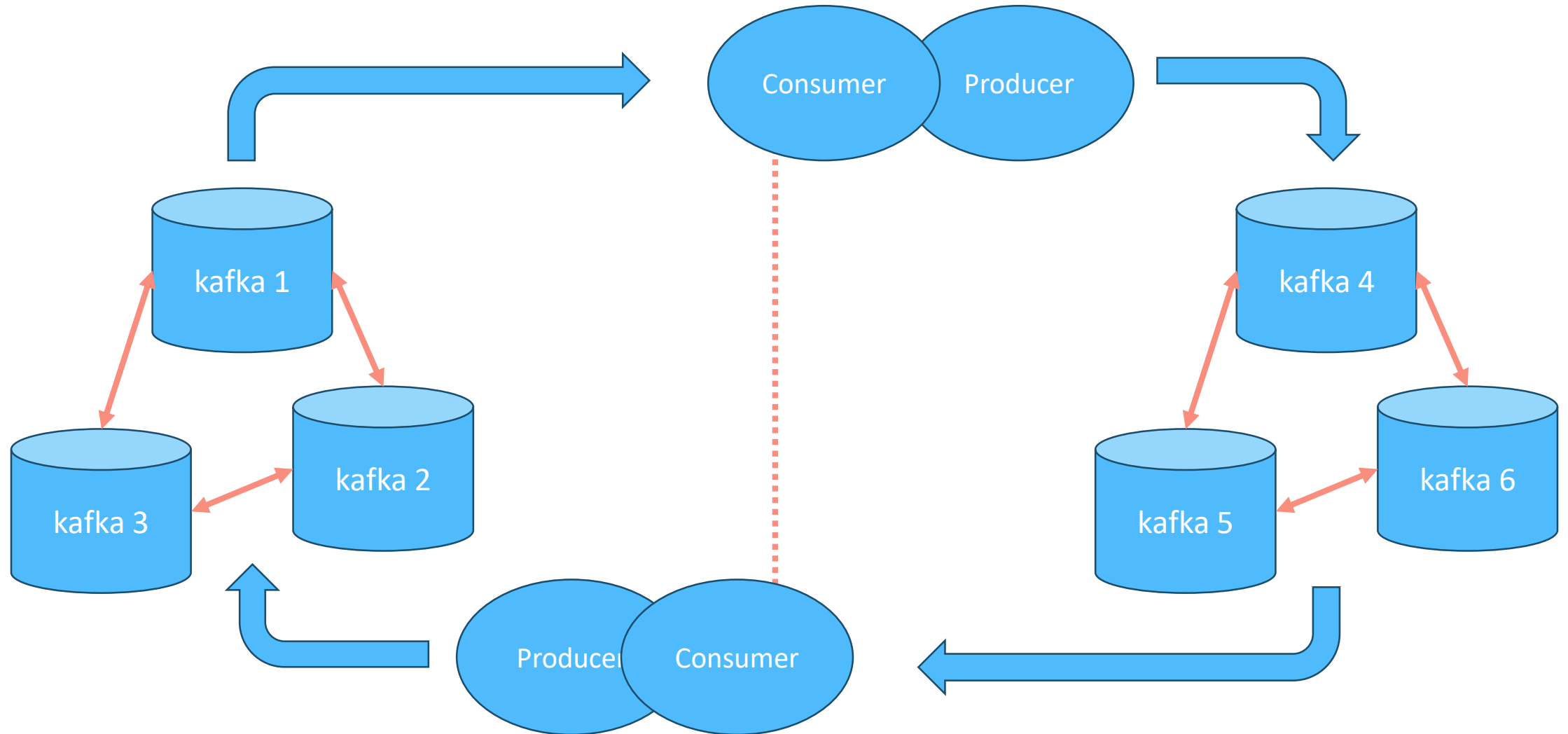
MirrorMaker



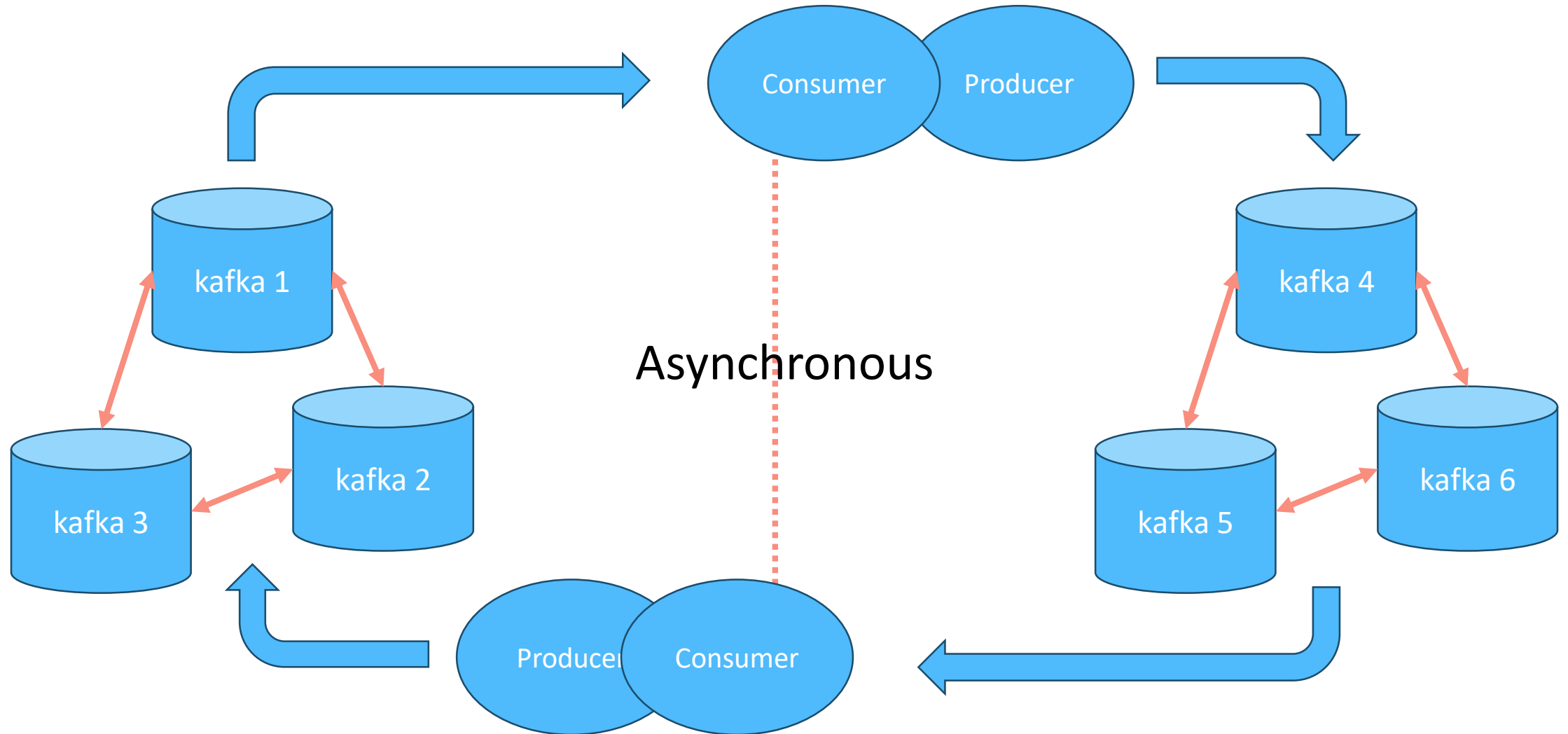
MirrorMaker



MirrorMaker



MirrorMaker



What is Kafka Mirroring?

The process of replicating data from one Kafka cluster (source) to another Kafka cluster (target)

Purpose:

- Data availability.
- Disaster recovery.
- Geographic distribution.
- Data locality for improved performance.

Setting Up MirrorMaker

```
# consumer.properties
bootstrap.servers=source-kafka:9092
exclude.internal.topics=true
client.id=mirror_maker_consumer
group.id=mirror_maker_consumer

# producer.properties
bootstrap.servers=target-kafka:9092
acks=all
batch.size=100
client.id=mirror_maker_producer
```

Setting Up MirrorMaker

```
bin/kafka-mirror-maker.sh \  
    --consumer.config consumer.properties \  
    --producer.config producer.properties \  
    --num.streams 3 \  
    --whitelist=".*"
```

Cycle Prevention Mechanisms

- **Topic Prefixing:**

Use different prefixes for topics in each cluster.

Example: cluster-a-topic and cluster-b-topic

- **Message Filtering:**

Use custom message handlers to filter out messages that have already been mirrored.

Cycle Prevention Mechanisms

```
./bin/kafka-mirror-maker.sh \  
  --consumer.config consumer-a.properties \  
  --producer.config producer-b.properties \  
  --whitelist="^cluster-a-.*" \  
  
./bin/kafka-mirror-maker.sh \  
  --consumer.config consumer-a.properties \  
  --producer.config producer-b.properties \  
  --message.handler=io.devhands.CustomHandler
```

Cycle Prevention CustomHandler

```
public class CustomHandler implements MirrorMakerMessageHandler {  
    @Override  
    public void handle(String key, String value, RecordContext recCtx) {  
        // Example: Add a header to mark messages that have been mirrored  
        if (recCtx.headers().lastHeader("mirrored-from-cluster-a") == null)  
{  
            context().forward(key, value);  
        }  
    }  
}
```

Homework

- Run Streams on top of the cluster from previous homework
- Create simple mapper for any of the simulated topic
- Implement simple aggregate processor
- Implement join processor

Homework

- Run Streams on top of the cluster from previous homework
 - Create simple mapper for any of the simulated topic
 - Implement simple aggregate processor
 - Implement join processor
-
- Could be done with Kafka Streams or any other approach

Demo